Assignment -3

Question-1:

P1.cs

```
using System;
public class Person(string name, int age)
{
   public string Name { get; set; } = name;
   public int Age { get; set; } = age;
}

class Program
{
   static void Main()
   {
      Person person1 = new("Leopold", 6);
      Console.WriteLine($"person1 Name = {person1.Name} Age = {person1.Age}");
       Person person2 = person1;
      person2.Name = "Molly";
      person2.Age = 16;
      Console.WriteLine($"person2 Name = {person2.Name} Age = {person2.Age}");
      Console.WriteLine($"person1 Name = {person1.Name} Age = {person1.Age}");
   }
}
```

P2.cs

```
using System;
namespace Example
```

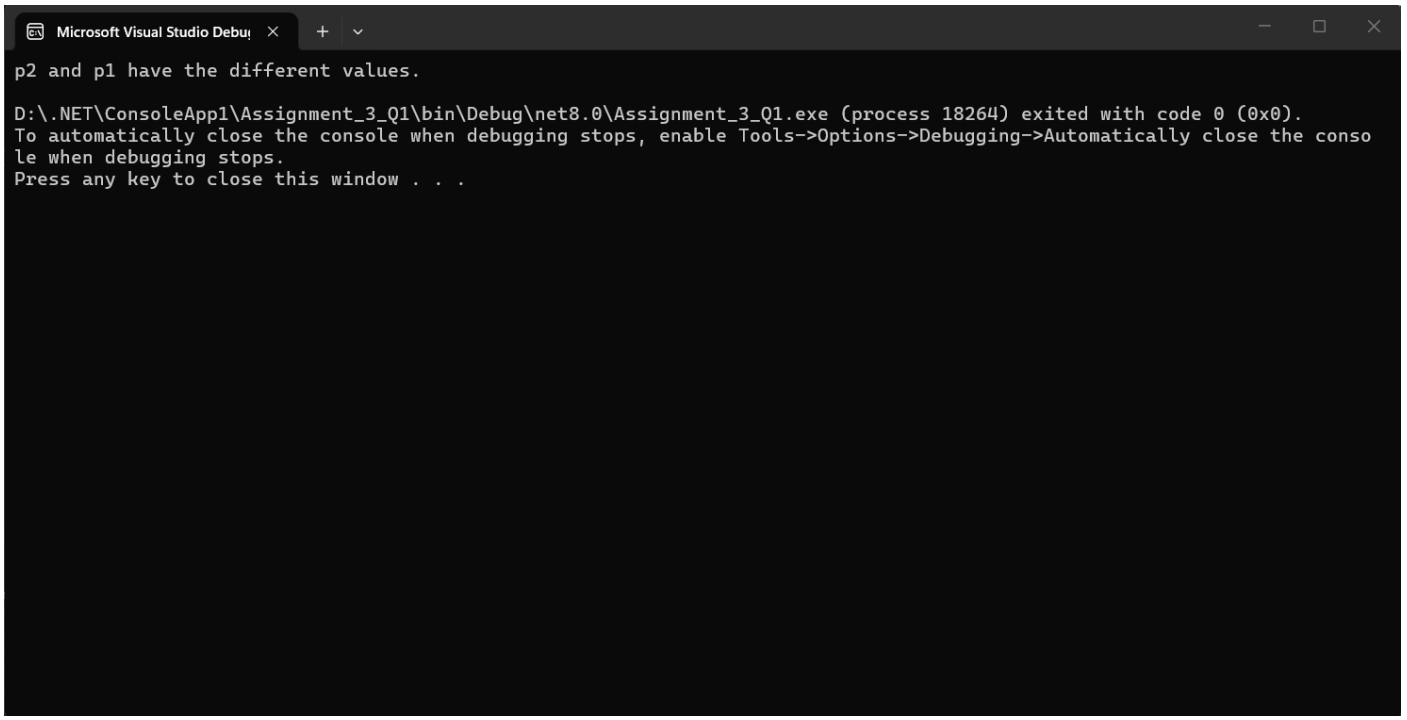
```
{
    public struct Person
    {
        public string Name;
        public int Age;
        public Person(string name, int age)
        {
            Name = name;
            Age = age;
        }
    }

    public class Application
    {
        static void Main()
        {
            Person p1 = new("Alex", 9);
            Console.WriteLine($"p1 Name = {p1.Name} Age = {p1.Age}");
            Person p2 = p1;
            p2.Name = "Spencer";
            p2.Age = 7;
            Console.WriteLine($"p2 Name = {p2.Name} Age = {p2.Age}");
            Console.WriteLine($"p1 Name = {p1.Name} Age = {p1.Age}");
        }
    }
}
```

Program.cs

```
using System;
namespace Example
{
    public struct Person
    {
        public string Name;
        public int Age;
        public Person(string name, int age)
        {
            Name = name;
            Age = age;
        }
    }

    public class Person(string name, int age)
    {
        public string Name { get; set; } = name;
        public int Age { get; set; } = age;
    }
    public class Application
    {
        static void Main()
        {
             Person p1 = new("Alex", 9);
            Console.WriteLine($"p1 Name = {p1.Name} Age = {p1.Age}");
            Person p2 = p1;
```

```
        p2.Name = "Spencer";
        p2.Age = 7;
        Console.WriteLine($"p2 Name = {p2.Name} Age = {p2.Age}");
        Console.WriteLine($"p1 Name = {p1.Name} Age = {p1.Age}");

        Person p1 = new("Wallace", 75);
        Person p2 = new("", 42);
        p2.Name = "Wallace";
        p2.Age = 75;

        if (p2.Equals(p1))
            Console.WriteLine("p2 and p1 have the same values.");
        else
            Console.WriteLine("p2 and p1 have the different values.");
    }
  }
}
```

Output :



Question-2:

Program.cs

```
using System;

namespace Assignment_3_Q2
{
    public class Project
    {
        public int ProjectId { get; set; }
        public string ProjectName { get; set; }

        public Project(int id, string name)
        {
            ProjectId = id;
            ProjectName = name;
        }

        public override string ToString()
        {
            return $"Project ID: {ProjectId}, Project Name: {ProjectName}";
        }
    }

    public class WorkItem
    {
        private static int currentID;
```

```
protected int ID { get; set; }
protected string Title { get; set; }
protected string Description { get; set; }
protected TimeSpan JobLength { get; set; }

// New property
public Project Project { get; set; }

static WorkItem()
{
    currentID = 0;
}

public WorkItem() { }

public WorkItem(string title, string desc, TimeSpan jobLen, Project project)
{
    ID = GetNextID();
    Title = title;
    Description = desc;
    JobLength = jobLen;
    Project = project;
}

protected int GetNextID() => ++currentID;

public override string ToString()
{
```

```
        return $"WorkItem ID: {ID}, Title: {Title}, Project: {Project.ProjectName}";
    }
}

public class ChangeRequest : WorkItem
{
    public int OriginalItemID { get; set; }

    public ChangeRequest(string title, string desc, TimeSpan jobLen,
                         Project project, int originalID)
        : base(title, desc, jobLen, project)
    {
        OriginalItemID = originalID;
    }

    public override string ToString()
    {
        return $"ChangeRequest ID: {OriginalItemID}, Title: {Title}, Project:
{Project.ProjectName}";
    }
}

class Program
{
    static void Main()
    {
        // Projects
        List<Project> projects = new List<Project>
```

```
{
    new Project(1, "Banking System"),
    new Project(2, "E-Commerce App"),
    new Project(3, "AI Chatbot")
};
  List<WorkItem> workItems = new List<WorkItem>
{
    new WorkItem("Login Module", "Create login", TimeSpan.FromHours(5),
projects[0]),
    new WorkItem("Payment Gateway", "Integrate payments",
TimeSpan.FromHours(8), projects[1]),
    new WorkItem("UI Design", "Design UI", TimeSpan.FromHours(6), projects[1]),
    new WorkItem("NLP Model", "Train model", TimeSpan.FromHours(10),
projects[2]),
    new WorkItem("Testing", "Unit testing", TimeSpan.FromHours(4), projects[0])
};
    List<ChangeRequest> changeRequests = new List<ChangeRequest>
{
    new ChangeRequest("Login Fix", "Fix validation", TimeSpan.FromHours(2),
projects[0], 1),
    new ChangeRequest("UI Change", "Dark mode", TimeSpan.FromHours(3),
projects[1], 3),
    new ChangeRequest("Model Update", "Improve accuracy",
TimeSpan.FromHours(5), projects[2], 4)
};


    Console.WriteLine("=== Projects ===");
    foreach (var p in projects)
        Console.WriteLine(p);
```

```
        Console.WriteLine("\n=== WorkItems ===");
        foreach (var w in workItems)
            Console.WriteLine(w);

        Console.WriteLine("\n=== Change Requests ===");
        foreach (var c in changeRequests)
            Console.WriteLine(c);
    }
  }
}
```

Output :

```
Microsoft Visual Studio Debu

=== Projects ===
Project ID: 1, Project Name: Banking System
Project ID: 2, Project Name: E-Commerce App
Project ID: 3, Project Name: AI Chatbot

=== WorkItems ===
WorkItem ID: 1, Title: Login Module, Project: Banking System
WorkItem ID: 2, Title: Payment Gateway, Project: E-Commerce App
WorkItem ID: 3, Title: UI Design, Project: E-Commerce App
WorkItem ID: 4, Title: NLP Model, Project: AI Chatbot
WorkItem ID: 5, Title: Testing, Project: Banking System

=== Change Requests ===
ChangeRequest ID: 1, Title: Login Fix, Project: Banking System
ChangeRequest ID: 3, Title: UI Change, Project: E-Commerce App
ChangeRequest ID: 4, Title: Model Update, Project: AI Chatbot

D:\.NET\ConsoleApp1\Assignment_3_Q2\bin\Debug\net8.0\Assignment_3_Q2.exe (process 27404) exited with code 0 (0x0).
Press any key to close this window . . .
```

Question-3:

Shape.cs

```
using System;
namespace Assignment_3_Q3
{
    public abstract class Shape
    {
        public abstract double GetArea();
        public abstract double GetPerimeter();

        public virtual void DisplayInfo()
        {
            Console.WriteLine("Shape Base Class");
        }
    }
}
```

Circle.cs

```
using System;
namespace Assignment_3_Q3
{
    public class Circle : Shape
    {
        public double Radius { get; set; }

        public Circle(double radius) => Radius = radius;

        public override double GetArea() => Math.PI * Radius * Radius;
```

```
        public override double GetPerimeter() => 2 * Math.PI * Radius;

        public override void DisplayInfo()
        {
            Console.WriteLine($"Circle: Radius {Radius}");
        }
    }
}
```

Rectangle.cs

```
using System;

namespace Assignment_3_Q3
{
    public sealed class Rectangle : Shape
    {
        public double Length { get; set; }
        public double Width { get; set; }

        public Rectangle(double l, double w)
        {
            Length = l;
            Width = w;
        }

        public override double GetArea() => Length * Width;
        public override double GetPerimeter() => 2 * (Length + Width);
```

```
        public override void DisplayInfo()
        {
            Console.WriteLine($"Rectangle: {Length}x{Width}");
        }
    }
}
```

Square.cs

```
using System;

namespace Assignment_3_Q3
{
    public class Square : Shape
    {
        public double Side { get; set; }

        public Square(double side) => Side = side;

        public override double GetArea() => Side * Side;
        public override double GetPerimeter() => 4 * Side;

        public new void DisplayInfo()
        {
            Console.WriteLine($"Square: Side {Side}");
        }
    }
}
```

Program.cs

```
using System;
using System.Collections.Generic;
namespace Assignment_3_Q3
{
    class Program
    {
        static void Main(string[] args)
        {
            List<Shape> shapes = new List<Shape>
            {
                new Circle(5),
                new Rectangle(4, 8),
                new Square(10)
            };
            foreach (var shape in shapes)
            {
                shape.DisplayInfo();
                Console.WriteLine($"Area: {shape.GetArea():F2}");
                Console.WriteLine($"Perimeter: {shape.GetPerimeter():F2}");
                Console.WriteLine();
            }
            Square s = new Square(6);
            s.DisplayInfo();
        }
    }
}
```

Output :

```
Microsoft Visual Studio Debu

Circle: Radius 5
Area: 78.54
Perimeter: 31.42

Rectangle: 4x8
Area: 32.00
Perimeter: 24.00

Shape Base Class
Area: 100.00
Perimeter: 40.00

Square: Side 6

D:\.NET\ConsoleApp1\Assignment_3_Q3\bin\Debug\net9.0\Assignment_3_Q3.exe (process 27040) exited with code 0 (0x0).
Press any key to close this window . . .
```

Assignment - 4

BankAccount.cs

```
using System;
using System.Collections.Generic;
using System.Text;

public class BankAccount
{
    private static int s_accountNumberSeed = 100000;

    public string Number { get; }
    public string Owner { get; set; }

    protected decimal Balance
    {
        get
        {
            decimal balance = 0;
            foreach (var item in _allTransactions)
                balance += item.Amount;
            return balance;
        }
    }

    private readonly decimal _minimumBalance;
    private List<Transaction> _allTransactions = new();
```

```
// Constructor chaining
public BankAccount(string name, decimal initialBalance)
    : this(name, initialBalance, 0) { }

public BankAccount(string name, decimal initialBalance, decimal minimumBalance)
{
    Number = s_accountNumberSeed.ToString();
    s_accountNumberSeed++;

    Owner = name;
    _minimumBalance = minimumBalance;

    if (initialBalance > 0)
        MakeDeposit(initialBalance, DateTime.Now, "Initial balance");
}

public void MakeDeposit(decimal amount, DateTime date, string note)
{
    ArgumentOutOfRangeException.ThrowIfNegativeOrZero(amount);
    _allTransactions.Add(new Transaction(amount, date, note));
}

public void MakeWithdrawal(decimal amount, DateTime date, string note)
{
    ArgumentOutOfRangeException.ThrowIfNegativeOrZero(amount);

    Transaction? overdraftTransaction =
        CheckWithdrawalLimit(Balance - amount < _minimumBalance);
```

```
        _allTransactions.Add(new Transaction(-amount, date, note));

        if (overdraftTransaction != null)
            _allTransactions.Add(overdraftTransaction);
    }

    protected virtual Transaction? CheckWithdrawalLimit(bool isOverdrawn)
    {
        if (isOverdrawn)
            throw new InvalidOperationException("Not sufficient funds for this
withdrawal");

        return null;
    }

    public virtual void PerformMonthEndTransactions() { }

    public string GetAccountHistory()
    {
        var report = new StringBuilder();

        report.AppendLine("Date\t\tAmount\tBalance\tNote");

        decimal balance = 0;
        foreach (var item in _allTransactions)
        {
            balance += item.Amount;
```

```
        report.AppendLine($"{item.Date:d}\t{item.Amount}\t{balance}\t{item.Note}");
    }
    return report.ToString();
  }
}
```

GiftCardAccount.cs

```
using System;
public class GiftCardAccount : BankAccount
{
  private readonly decimal _monthlyDeposit;

  public GiftCardAccount(string name, decimal initialBalance, decimal
monthlyDeposit = 0)
    : base(name, initialBalance)
  {
    _monthlyDeposit = monthlyDeposit;
  }

  public override void PerformMonthEndTransactions()
  {
    if (_monthlyDeposit != 0)
    {
      MakeDeposit(_monthlyDeposit, DateTime.Now, "Add monthly deposit");
    }
  }
}
```

IntrestEarningAccount.cs

```
using System;

public class InterestEarningAccount : BankAccount
{
    public InterestEarningAccount(string name, decimal initialBalance)
        : base(name, initialBalance) { }

    public override void PerformMonthEndTransactions()
    {
        if (Balance > 500)
        {
            decimal interest = Balance * 0.02m;
            MakeDeposit(interest, DateTime.Now, "Apply monthly interest");
        }
    }
}
```

LineOfCreditAcccount.cs

```
using System;

public class LineOfCreditAccount : BankAccount
{
    public LineOfCreditAccount(string name, decimal initialBalance, decimal
creditLimit)
        : base(name, initialBalance, -creditLimit) { }

    public override void PerformMonthEndTransactions()
```

```
    {
        if (Balance < 0)
        {
            decimal interest = -Balance * 0.07m;
            MakeWithdrawal(interest, DateTime.Now, "Charge monthly interest");
        }
    }

    protected override Transaction? CheckWithdrawalLimit(bool isOverdrawn) =>
        isOverdrawn
        ? new Transaction(-20, DateTime.Now, "Apply overdraft fee")
        : null;
}
```

Transction.cs

```
using System;
public class Transaction
{
    public decimal Amount { get; }
    public DateTime Date { get; }
    public string Note { get; }
    public Transaction(decimal amount, DateTime date, string note)
    {
        Amount = amount;
        Date = date;
        Note = note;
    }
}
```

Program.cs

```
using System;

class Program
{
    static void Main()
    {
        var giftCard = new GiftCardAccount("Gift Card", 100, 50);
        giftCard.MakeWithdrawal(20, DateTime.Now, "Coffee");
        giftCard.MakeWithdrawal(50, DateTime.Now, "Groceries");
        giftCard.PerformMonthEndTransactions();
        giftCard.MakeDeposit(25, DateTime.Now, "Extra money");
        Console.WriteLine("=== Gift Card Account ===");
        Console.WriteLine(giftCard.GetAccountHistory());

        var savings = new InterestEarningAccount("Savings", 10000);
        savings.MakeDeposit(750, DateTime.Now, "Salary");
        savings.MakeWithdrawal(250, DateTime.Now, "Bills");
        savings.PerformMonthEndTransactions();
        Console.WriteLine("=== Interest Account ===");
        Console.WriteLine(savings.GetAccountHistory());

        var credit = new LineOfCreditAccount("Line of Credit", 0, 2000);
        credit.MakeWithdrawal(1000, DateTime.Now, "Advance");
        credit.MakeDeposit(100, DateTime.Now, "Partial payment");
        credit.MakeWithdrawal(5000, DateTime.Now, "Emergency");
        credit.PerformMonthEndTransactions();
```

```
        Console.WriteLine("=== Line of Credit Account ===");
        Console.WriteLine(credit.GetAccountHistory());
    }
}
```

Output :

```
Microsoft Visual Studio Debu

=== Gift Card Account ===
Date            Amount  Balance Note
22-02-2026      100     100     Initial balance
22-02-2026      -20     80      Coffee
22-02-2026      -50     30      Groceries
22-02-2026      50      80      Add monthly deposit
22-02-2026      25      105     Extra money

=== Interest Account ===
Date            Amount  Balance Note
22-02-2026      10000   10000   Initial balance
22-02-2026      750     10750   Salary
22-02-2026      -250    10500   Bills
22-02-2026      210.00  10710.00        Apply monthly interest

=== Line of Credit Account ===
Date            Amount  Balance Note
22-02-2026      -1000   -1000   Advance
22-02-2026      100     -900    Partial payment
22-02-2026      -5000   -5900   Emergency
22-02-2026      -20     -5920   Apply overdraft fee
22-02-2026      -414.40 -6334.40        Charge monthly interest
22-02-2026      -20     -6354.40        Apply overdraft fee


D:\.NET\ConsoleApp1\Assignment_4\bin\Debug\net8.0\Assignment_4.exe (process 25244) exited with code 0 (0x0).
Press any key to close this window . . .
```

Assignment - 5

FileLogger.cs

```
using System.IO;


namespace Assignment_5
{
    public class FileLogger
    {
        private readonly string _path;


        public FileLogger(string path)
        {
            _path = path;
            Logger.MessageLogged += OnMessageLogged;
        }


        private void OnMessageLogged(object? sender, LoggerEventArgs e)
        {
            try
            {
                string formattedMsg =
$"{e.Timestamp:G}\t{e.Severity}\t{e.Component}\t{e.Message}";
                File.AppendAllLines(_path, new[] { formattedMsg });
            }
            catch { }
        }
```

```
        public void Detach() => Logger.MessageLogged -= OnMessageLogged;
    }
}
```

Logger.cs

```
using System;

namespace Assignment_5
{
    public static class Logger
    {
        public static event EventHandler<LoggerEventArgs>? MessageLogged;
        public static Severity LogLevel { get; set; } = Severity.Warning;

        public static void LogMessage(Severity s, string component, string msg)
        {
            if (s < LogLevel) return;
            MessageLogged?.Invoke(null, new LoggerEventArgs(s, component, msg));
        }
    }
}
```

LoggerEventArgs.cs

```
using System;

namespace Assignment_5
{
```

```
    public class LoggerEventArgs : EventArgs
    {
        public Severity Severity { get; }
        public string Component { get; }
        public string Message { get; }
        public DateTime Timestamp { get; }

        public LoggerEventArgs(Severity s, string comp, string msg)
        {
            Severity = s;
            Component = comp;
            Message = msg;
            Timestamp = DateTime.Now;
        }
    }
}
```

Severity.cs

```
namespace Assignment_5
{
    public enum Severity
    {
        Verbose,
        Trace,
        Information,
        Warning,
        Error,
```

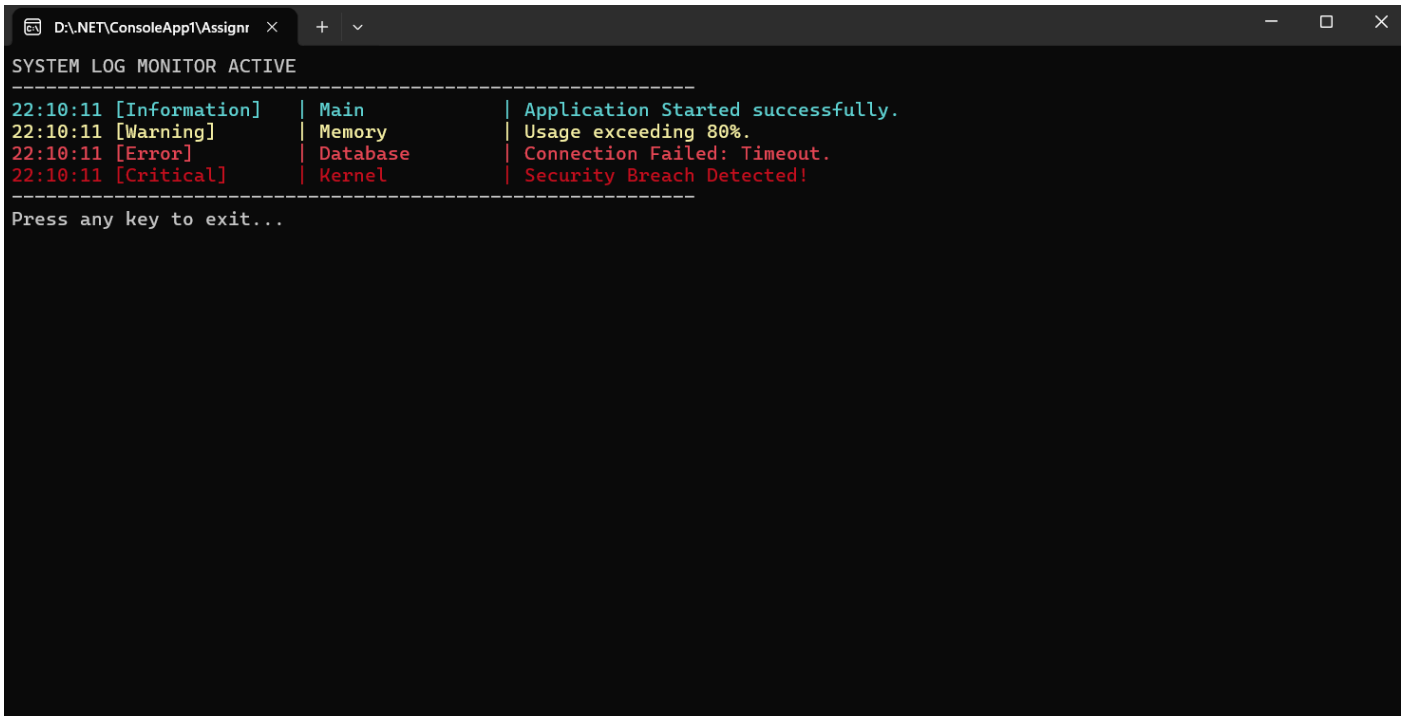
```
        Critical
    }
}
```

Program.cs

```
using System;
namespace Assignment_5
{
    class Program
    {
        static void Main(string[] args)
        {
            Logger.LogLevel = Severity.Information;

            Logger.MessageLogged += (sender, e) =>
            {
                Console.ForegroundColor = e.Severity switch
                {
                    Severity.Critical => ConsoleColor.DarkRed,
                    Severity.Error => ConsoleColor.Red,
                    Severity.Warning => ConsoleColor.Yellow,
                    Severity.Information => ConsoleColor.Cyan,
                    _ => ConsoleColor.Gray
                };
                string timestamp = e.Timestamp.ToString("HH:mm:ss");
                string severity = $"[{e.Severity}]".PadRight(15);
                string component = e.Component.PadRight(15);
```

```
            Console.WriteLine($"{timestamp} {severity} | {component} | {e.Message}");
            Console.ResetColor();
        };

        Console.WriteLine("SYSTEM LOG MONITOR ACTIVE");
        Console.WriteLine(new string('-', 60));
        Logger.LogMessage(Severity.Information, "Main", "Application Started
successfully.");
        Logger.LogMessage(Severity.Warning, "Memory", "Usage exceeding 80%.");
        Logger.LogMessage(Severity.Error, "Database", "Connection Failed: Timeout.");
        Logger.LogMessage(Severity.Critical, "Kernel", "Security Breach Detected!");
        Console.WriteLine(new string('-', 60));
        Console.WriteLine("Press any key to exit...");
        Console.ReadKey();
    }
  }
}
```

Output :



Assignment - 6

TodoItemsController.cs

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using TodoApi.Models;

namespace TodoApi.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class TodoItemsController : ControllerBase
    {
        private readonly TodoContext _context;

        public TodoItemsController(TodoContext context)
        {
            _context = context;
        }

        // GET: api/TodoItems (READ ALL)
        [HttpGet]
        public async Task<ActionResult<IEnumerable<TodoItem>>> GetTodoItems()
        {
            return await _context.TodoItems.ToListAsync();
        }

        // GET: api/TodoItems/5 (READ ONE)
```

```
    [HttpGet("{id}")]
    public async Task<ActionResult<TodoItem>> GetTodoItem(long id)
    {
        var todoItem = await _context.TodoItems.FindAsync(id);
        if (todoItem == null) return NotFound();
        return todoItem;
    }

    // POST: api/TodoItems (CREATE)
    [HttpPost]
    public async Task<ActionResult<TodoItem>> PostTodoItem(TodoItem todoItem)
    {
        _context.TodoItems.Add(todoItem);
        await _context.SaveChangesAsync();
        return CreatedAtAction(nameof(GetTodoItem), new { id = todoItem.Id },
todoItem);
    }

    // PUT: api/TodoItems/5 (UPDATE)
    [HttpPut("{id}")]
    public async Task<IActionResult> PutTodoItem(long id, TodoItem todoItem)
    {
        if (id != todoItem.Id) return BadRequest();

        _context.Entry(todoItem).State = EntityState.Modified;
        await _context.SaveChangesAsync();
        return NoContent();
    }
```

```
        // DELETE: api/TodoItems/5 (DELETE)
        [HttpDelete("{id}")]
        public async Task<IActionResult> DeleteTodoItem(long id)
        {
            var todoItem = await _context.TodoItems.FindAsync(id);
            if (todoItem == null) return NotFound();

            _context.TodoItems.Remove(todoItem);
            await _context.SaveChangesAsync();
            return NoContent();
        }
    }
}
```

20260209110314_AddTodoTable.cs

```
using Microsoft.EntityFrameworkCore.Migrations;

#nullable disable

namespace TodoApi.Migrations
{
    /// <inheritdoc />
    public partial class AddTodoTable : Migration
    {
        /// <inheritdoc />
        protected override void Up(MigrationBuilder migrationBuilder)
        {
```

```
            migrationBuilder.CreateTable(
                name: "TodoItems",
                columns: table => new
                {
                    Id = table.Column<long>(type: "bigint", nullable: false)
                        .Annotation("SqlServer:Identity", "1, 1"),
                    Name = table.Column<string>(type: "nvarchar(max)", nullable: true),
                    IsComplete = table.Column<bool>(type: "bit", nullable: false)
                },
                constraints: table =>
                {
                    table.PrimaryKey("PK_TodoItems", x => x.Id);
                });
        }

        /// <inheritdoc />
        protected override void Down(MigrationBuilder migrationBuilder)
        {
            migrationBuilder.DropTable(
                name: "TodoItems");
        }
    }
}
```

TodoContext.cs

```
using Microsoft.EntityFrameworkCore;
namespace TodoApi.Models;
```

```
public class TodoContext : DbContext
{
    public TodoContext(DbContextOptions<TodoContext> options)
        : base(options)
    {
    }
    public DbSet<TodoItem> TodoItems { get; set; } = null!;
}
```

Todoitems.cs

```
namespace TodoApi.Models;
public class TodoItem
{
    public long Id { get; set; }
    public string? Name { get; set; }
    public bool IsComplete { get; set; }
}
```

Program.cs

```
using Microsoft.EntityFrameworkCore;
using TodoApi.Models;
var builder = WebApplication.CreateBuilder(args);
// Add Controllers
builder.Services.AddControllers();
// Add Swagger
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();
```

```
// Get Connection String
var connectionString =
builder.Configuration.GetConnectionString("DefaultConnection");


// Register DbContext (DI)
builder.Services.AddDbContext<TodoContext>(opt =>
    opt.UseSqlServer(connectionString));


// Add CORS
builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowAll", policy =>
    {
        policy.AllowAnyOrigin()
            .AllowAnyMethod()
            .AllowAnyHeader();
    });
});
var app = builder.Build();
// Swagger UI
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}
app.UseHttpsRedirection();
app.UseCors("AllowAll"); // Important for React
```

```
app.UseAuthorization();
app.MapControllers();
app.Run();
```

App.js

```
import React, { useEffect, useState } from "react";
import "./App.css";

function App() {

  const [todos, setTodos] = useState([]);
  const [newTask, setNewTask] = useState("");

  const [editId, setEditId] = useState(null);
  const [editText, setEditText] = useState("");

  const API_URL = "https://localhost:7271/api/TodoItems";

  // Load data
  const loadTodos = () => {
    fetch(API_URL)
      .then(res => res.json())
      .then(data => setTodos(data))
      .catch(err => console.error(err));
  };

  useEffect(() => {
```

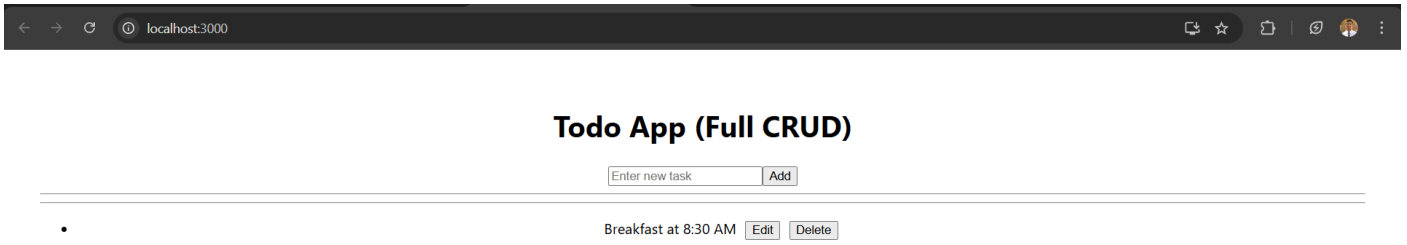
```
    loadTodos();
}, []);

// Add
const addTodo = () => {

    if (newTask.trim() === "") return;

    fetch(API_URL, {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({
            name: newTask,
            isComplete: false
        })
    })
        .then(() => {
            setNewTask("");
            loadTodos();
        });
};

// Delete
const deleteTodo = (id) => {

    fetch(`${API_URL}/${id}`, {
        method: "DELETE"
    })
```

```
        .then(() => loadTodos());
};

// Start Edit
const startEdit = (todo) => {
    setEditId(todo.id);
    setEditText(todo.name);
};

// Update
const updateTodo = () => {

    fetch(`${API_URL}/${editId}`, {
        method: "PUT",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({
            id: editId,
            name: editText,
            isComplete: false
        })
    })
        .then(() => {
            setEditId(null);
            setEditText("");
            loadTodos();
        });
};
```

```
return (
  <div style={{ padding: "40px", textAlign: "center" }}>

    <h1>Todo App (Full CRUD)</h1>

    {/* Add */}
    <input
      type="text"
      placeholder="Enter new task"
      value={newTask}
      onChange={(e) => setNewTask(e.target.value)}
    />

    <button onClick={addTodo}>Add</button>

    <hr />

    {/* Edit */}
    {editId && (
      <div>
        <input
          value={editText}
          onChange={(e) => setEditText(e.target.value)}
        />
        <button onClick={updateTodo}>Update</button>
        <button onClick={() => setEditId(null)}>Cancel</button>
      </div>
    )}
```

```
        <hr />

        {/* List */}
        <ul>
          {todos.map((t) => (
            <li key={t.id}>
              {t.name}

              <button
                onClick={() => startEdit(t)}
                style={{ marginLeft: "10px" }}
              >
                Edit
              </button>
              <button
                onClick={() => deleteTodo(t.id)}
                style={{ marginLeft: "10px" }}
              >
                Delete
              </button>
            </li>
          ))}
        </ul>
      </div>
    );
}
export default App;
```

Output :


localhost:3000
Todo App (Full CRUD)
Enter new task
Add
Breakfast at 8:30 AM
Edit
Delete